Knowledge Representation for Employee Leave Approval

AIMLZG557 Artificial and Computational Intelligence · Situated Learning Assignment · Question 2 (individual) · Section B: Student Submission

B.1 Submission Form

Name	Vanshika Sangal
ID No	2025AG05381
Industry / Domain	Data Engineering / AI Data Services (IT) – Human Resources Operations
Organisation Type	Innodata – technology company with project-based delivery teams
Instructor prompt	Employee leave approval governed by HR policy (rule-based decision process)

B.2 Anchor (1 mark)

Workplace process: Employee Leave Approval. At Innodata, a data engineering company that works in project-based delivery teams against client deadlines, every leave request (planned earned leave, sick leave or emergency leave) is checked against written HR policy before it is approved.

Why it matters. A delivery team that loses people during a client go-live can miss a deadline, while a wrongly denied leave hurts the employee and trust in HR. The decisions are repetitive and rule-governed, and they must be consistent and explainable. That makes the process a good fit for a logic-based Knowledge Base (KB).

How a leave request flows:

- The employee raises the request in the HR system: leave type, dates and number of working days.
- The reporting manager reviews it against the policy: eligibility, notice, team coverage and project freeze.
- Leave longer than 5 working days also goes to HR for approval.
- The outcome, Approved or Denied, is recorded and the employee is notified.

The policy parts:

- **Eligibility:** only confirmed employees (probation completed) with enough leave balance can take planned leave.
- **Planned leave:** at least 3 working days' notice, adequate team coverage, and the dates must not fall in a project freeze (client delivery or go-live window).
- **Emergency leave:** exempt from notice, coverage and freeze checks, but still needs balance.
- **Sick leave:** needs a medical certificate and is not blocked by freeze or balance checks (it has its own quota).
- **Escalation:** leave longer than 5 working days needs HR approval in addition to manager approval.
- **Closure:** every decision (approve or deny) is communicated to the employee.

Who is involved: the employee, the reporting manager, the HR manager and the HR system. In the example: employees Asha, Ravi, Meera, Karan and Neha; managers Suresh and Priya; HR manager Anita; six leave requests L1 to L6.

Where the facts come from: the HR system (employee status, balance, leave requests), the project calendar (freeze windows), the team roster (coverage) and the attendance records (medical certificates).

Names and request numbers in this document are illustrative. No real employee data is used.

B.3 Apply (2 marks)

The policy gives 13 business rules (B.3.1). They are written in propositional logic (B.3.2), extended to predicate logic (B.3.3), turned into a knowledge base of **41 facts** and 13 inference rules (B.3.4), and used to answer five workplace queries (B.3.5).

B.3.1 Business rules

Rule	Business rule
R1	A confirmed employee with enough balance is eligible for planned leave.
R2	Eligible planned leave with notice, adequate coverage and no freeze meets manager approval.
R3	Emergency leave with enough balance meets manager approval, ignoring notice, coverage and freeze.
R4	Sick leave with a medical certificate meets manager approval.
R5	Manager-approved leave longer than 5 days needs HR approval.
R6	Leave needing HR approval is approved once HR approves.
R7	Manager-approved leave of 5 days or less is approved.
R8	Planned leave in a freeze, without coverage or without notice is denied (unless emergency or sick).
R9	Leave without balance is denied (unless sick leave).
R10	Unconfirmed (probation) employees are denied planned leave.
R11	Sick leave without a medical certificate is denied.
R12	If HR approval is needed but HR has not approved, the request is denied.
R13	Every approved or denied request is notified to the employee.

B.3.2 Propositional logic

Each proposition is a statement about the one leave request under review, and it is either true or false.

Symbol	Meaning	Symbol	Meaning
Conf	The employee has completed probation	Bal	The employee has enough leave balance
Notice	The request was raised at least 3 working days ahead	Cov	Team coverage is adequate
Freeze	The dates fall inside a project freeze	Sick	The request is sick leave
Cert	A valid medical certificate is attached	Emerg	The request is emergency leave
Long	The leave is longer than 5 working days	Elig	The employee is eligible for planned leave (derived)
MgrOK	The manager's approval condition is met (derived)	NeedHR	HR approval is required (derived)
HROK	HR has approved the request	Appr	The request is approved (derived)
Deny	The request is denied (derived)	Notify	The employee is notified (derived)

The 13 business rules in propositional logic ($\wedge$ = and, $\vee$ = or, $\neg$ = not, $\rightarrow$ = implies):

Rule	Propositional formula
R1	$Conf \wedge Bal \rightarrow Elig$
R2	$Elig \wedge Notice \wedge Cov \wedge \neg Freeze \rightarrow MgrOK$
R3	$Emerg \wedge Bal \rightarrow MgrOK$
R4	$Sick \wedge Cert \rightarrow MgrOK$
R5	$MgrOK \wedge Long \rightarrow NeedHR$
R6	$NeedHR \wedge HROK \rightarrow Appr$
R7	$MgrOK \wedge \neg Long \rightarrow Appr$

Rule	Propositional formula
R8	(Freeze ∨ ¬Cov ∨ ¬Notice) ∧ ¬Emerg ∧ ¬Sick → Deny
R9	¬Bal ∧ ¬Sick → Deny
R10	¬Conf ∧ ¬Sick ∧ ¬Emerg → Deny
R11	Sick ∧ ¬Cert → Deny
R12	NeedHR ∧ ¬HROK → Deny
R13	(Appr ∨ Deny) → Notify

The 16 symbols describe only one request. To handle Asha's and Ravi's requests together we would need copies such as Conf_Asha, Conf_Ravi, Bal_Asha_L1, … and the 13 rules would have to be rewritten for every request. Predicate logic removes this problem.

B.3.3 Predicate logic

The same rules extended with constants, variables, predicates and quantifiers.

Constants

Type	Constants
Employees	Asha, Ravi, Meera, Karan, Neha
Managers	Suresh, Priya
HR manager	Anita
Leave requests	L1, L2, L3, L4, L5, L6
Numbers	durations in working days (1, 2, 3, 8, …)

Variables: x, y employees; r, s leave requests; h an HR person; t a team; d a duration in days.

Predicates (the first group is stored as facts in the KB; the second group is derived by the rules)

Predicate	Meaning	Kind
Confirmed(x)	Employee x has completed probation	Fact
SubmittedBy(r, x)	Request r was raised by employee x	Fact
Manages(m, x)	Manager m is the reporting manager of x (used to name the approver in Query 2; no rule P1–P13 needs it)	Fact
HRManager(h)	h is an HR manager	Fact
HasBalance(x, r)	x has enough balance for request r	Fact
Duration(r, d)	Request r is for d working days	Fact
NoticeGiven(r), Covered(r), InFreeze(r)	Notice rule met; team coverage adequate; dates inside a freeze window	Fact
Sick(r), Emergency(r), CertAttached(r)	Type of request; medical certificate attached	Fact
HRApproves(h, r)	HR manager h has approved request r	Fact
Eligible(x, r), MgrApproved(r), NeedsHR(r)	Eligibility; manager condition met; HR approval needed	Derived
Approved(r), Denied(r), Notified(x, r)	Final outcomes and notification	Derived

Employee(x) and Request(r) are only type labels for the variables x and r. They are not stored as facts and no rule uses them.

Rules in predicate logic

Rule	Predicate logic formula
P1	∀x ∀r [Confirmed(x) ∧ SubmittedBy(r,x) ∧ HasBalance(x,r) → Eligible(x,r)]
P2	∀x ∀r [Eligible(x,r) ∧ NoticeGiven(r) ∧ Covered(r) ∧ ¬InFreeze(r) → MgrApproved(r)]
P3	∀x ∀r [SubmittedBy(r,x) ∧ Emergency(r) ∧ HasBalance(x,r) → MgrApproved(r)]
P4	∀r [Sick(r) ∧ CertAttached(r) → MgrApproved(r)]
P5	∀r ∀d [MgrApproved(r) ∧ Duration(r,d) ∧ d > 5 → NeedsHR(r)]
P6	∀r ∀h [NeedsHR(r) ∧ HRManager(h) ∧ HRApproves(h,r) → Approved(r)]
P7	∀r ∀d [MgrApproved(r) ∧ Duration(r,d) ∧ d ≤ 5 → Approved(r)]
P8	∀r [(InFreeze(r) ∨ ¬Covered(r) ∨ ¬NoticeGiven(r)) ∧ ¬Emergency(r) ∧ ¬Sick(r) → Denied(r)]
P9	∀x ∀r [SubmittedBy(r,x) ∧ ¬HasBalance(x,r) ∧ ¬Sick(r) → Denied(r)]
P10	∀x ∀r [SubmittedBy(r,x) ∧ ¬Confirmed(x) ∧ ¬Sick(r) ∧ ¬Emergency(r) → Denied(r)]
P11	∀r [Sick(r) ∧ ¬CertAttached(r) → Denied(r)]
P12	∀r [NeedsHR(r) ∧ ¬∃h (HRManager(h) ∧ HRApproves(h,r)) → Denied(r)]
P13	∀x ∀r [SubmittedBy(r,x) ∧ (Approved(r) ∨ Denied(r)) → Notified(x,r)]

Extension rules (specified only; not part of the 13-rule KB and not run in the program). These two rules show something only predicate logic can express, because they relate several employees or requests. They use two extra predicates, MemberOf(x, t) (x belongs to team t) and Overlaps(r, s) (the dates of requests r and s overlap). The KB stores no facts for these two predicates, so P14 and P15 are never triggered by the six test requests.

Rule	Predicate logic formula	Meaning
P14	∀x ∀r ∀s [SubmittedBy(r,x) ∧ SubmittedBy(s,x) ∧ r ≠ s ∧ Overlaps(r,s) ∧ Approved(r) → Denied(s)]	An employee cannot have two approved leaves on overlapping dates.
P15	∀r ∀x ∀t [SubmittedBy(r,x) ∧ MemberOf(x,t) ∧ ∃y ∃s (y ≠ x ∧ MemberOf(y,t) ∧ SubmittedBy(s,y) ∧ Approved(s) ∧ Overlaps(r,s)) → ¬Covered(r)]	If a teammate already has approved leave on the same dates, coverage is inadequate.

How the quantifiers are used:

- ∀ over x and r makes one rule cover every employee and every request.
- ∃h in P12 means "some HR manager has approved" (it is used in the program). ∃y ∃s in the extension rule P15 means "some teammate is already on leave".
- P5 and P7 use a numeric comparison (d > 5, d ≤ 5) inside the rule, and the extension rule P14 uses an inequality (r ≠ s).

B.3.4 Knowledge base

The knowledge base holds **41 facts** and **13 inference rules** (P1 to P13). P14 and P15 are optional extensions and are not counted. I also coded the KB in Python (Appendix A) and ran it, so every answer in B.3.5 was checked by the program (41 base facts + 21 derived facts = 62 facts in total).

Leave requests in the knowledge base

Request	Employee (manager)	What it is	Days
L1	Asha (Suresh)	Planned leave, notice given, team covered	2
L2	Ravi (Suresh)	Planned leave, notice given, team covered; HR has approved	8
L3	Meera (Suresh)	Sick leave with medical certificate; no balance fact	3
L4	Karan (Priya)	Planned leave; Karan is on probation	3

Request	Employee (manager)	What it is	Days
L5	Neha (Priya)	Planned leave that falls in a project freeze	3
L6	Asha (Suresh)	Emergency leave that falls in a project freeze	1

Facts (41), in groups:

Facts	Group and facts	Count
F1–F4	Employee status: Confirmed(Asha), Confirmed(Ravi), Confirmed(Meera), Confirmed(Neha). Karan is on probation, so no Confirmed(Karan) fact is stored.	4
F5–F10	Organisation: Manages(Suresh, Asha), Manages(Suresh, Ravi), Manages(Suresh, Meera), Manages(Priya, Karan), Manages(Priya, Neha); HRManager(Anita)	6
F11–F16	Requests: SubmittedBy(L1, Asha), SubmittedBy(L2, Ravi), SubmittedBy(L3, Meera), SubmittedBy(L4, Karan), SubmittedBy(L5, Neha), SubmittedBy(L6, Asha)	6
F17–F22	Durations: Duration(L1, 2), Duration(L2, 8), Duration(L3, 3), Duration(L4, 3), Duration(L5, 3), Duration(L6, 1)	6
F23–F27	Balance: HasBalance(Asha, L1), HasBalance(Ravi, L2), HasBalance(Karan, L4), HasBalance(Neha, L5), HasBalance(Asha, L6)	5
F28–F31	Notice: NoticeGiven(L1), NoticeGiven(L2), NoticeGiven(L4), NoticeGiven(L5)	4
F32–F35	Coverage: Covered(L1), Covered(L2), Covered(L4), Covered(L5)	4
F36–F37	Project calendar: InFreeze(L5), InFreeze(L6)	2
F38–F40	Leave type: Sick(L3), CertAttached(L3), Emergency(L6)	3
F41	HR action: HRApproves(Anita, L2)	1
	Total	41

Facts not listed (for example NoticeGiven(L6) or Confirmed(Karan)) are treated as false (closed-world assumption).

Inference rules (13). The rules P1 to P13 above are the inference rules. Variables are universally quantified. Reasoning uses forward chaining (apply rules to known facts until nothing new is derived) and backward chaining (start from the query and look for supporting facts). In the program, backward chaining is implemented as a proof trace: it walks back from the goal through the derivations recorded during forward chaining and prints the supporting tree. In B.3.5 the goal-first reasoning is also worked out by hand.

B.3.5 Workplace queries

Five questions HR asks before closing the leave cycle. Query 1 and Query 3 use forward chaining (start from the facts), Query 2 and Query 4 use backward chaining (start from the goal; the program prints these as proof traces over the forward-chained derivations), and Query 5 compares two requests that both fall in a project freeze.

Query 1: Is Asha's request L1 (2 days) approved?

Method: forward chaining. Rules fire on known facts until Approved(L1) appears.

Step	New fact	Rule	Because of
1	Eligible(Asha, L1)	P1	SubmittedBy(L1, Asha) F11, Confirmed(Asha) F1, HasBalance(Asha, L1) F23
2	MgrApproved(L1)	P2	Eligible(Asha, L1), NoticeGiven(L1) F28, Covered(L1) F32; InFreeze(L1) is not a fact, so ¬InFreeze(L1)
3	Approved(L1)	P7	MgrApproved(L1), Duration(L1, 2) F17, 2 ≤ 5
4	Notified(Asha, L1)	P13	SubmittedBy(L1, Asha), Approved(L1)

Answer: yes. No denial rule fires: P8 needs a freeze, no coverage or no notice (none is true), P9 needs no balance and P10 needs unconfirmed status. NeedsHR(L1) is not derived because 2 ≤ 5, so no HR approval is needed.

Query 2: Ravi's request L2 is for 8 days. Is it approved, and who must approve it?

Method: backward chaining. Start from the goal Approved(L2) and work back to facts.

Step	Goal	How it is resolved	Result
1	Approved(L2)	P6 and P7 conclude Approved. P7 needs d ≤ 5, but Duration(L2, 8) makes 8 ≤ 5 false, so P7 fails. P6 gives the subgoals NeedsHR(L2) and an HR approver.	P6 chosen
2	NeedsHR(L2)	P5 needs MgrApproved(L2), Duration(L2, 8) F18 and 8 > 5.	subgoal MgrApproved
3	MgrApproved(L2)	P2 needs Eligible(Ravi, L2), NoticeGiven(L2) F29, Covered(L2) F33 and ¬InFreeze(L2) (no such fact).	✓ proved
4	Eligible(Ravi, L2)	P1: Confirmed(Ravi) F2, SubmittedBy(L2, Ravi) F12, HasBalance(Ravi, L2) F24.	✓ proved
5	∃h HRManager(h) ∧ HRApproves(h, L2)	HRManager(Anita) F10 and HRApproves(Anita, L2) F41, with h = Anita.	✓ proved

Answer: yes. L2 needs two approvals: the manager condition (Suresh, who manages Ravi per F6) and HR (Anita). If F41 were missing, P12 would give Denied(L2).

Query 3: Is Meera's sick leave L3 approved even though she has no leave balance fact?

Method: forward chaining, then a check of which denial rules could fire.

Step	New fact	Rule	Because of
1	MgrApproved(L3)	P4	Sick(L3) F38, CertAttached(L3) F39
2	Approved(L3)	P7	MgrApproved(L3), Duration(L3, 3) F19, 3 ≤ 5
3	Notified(Meera, L3)	P13	SubmittedBy(L3, Meera), Approved(L3)

Rule that could deny L3	Why it cannot fire
P8, P9, P10	Each contains ¬Sick(r) in its conditions, and Sick(L3) is true.
P11	Needs ¬CertAttached(L3), but F39 says the certificate is attached.
P2 (planned route)	Needs Eligible(Meera, L3), which needs HasBalance. It is not derived, but it is not needed: P4 gives MgrApproved(L3) directly.

Answer: yes. Sick leave with a certificate is approved and is not blocked by the balance rule.

Query 4: Can Karan's request L4 be approved? If not, why?

Method: backward chaining. The goal Approved(L4) is tried first, and then Denied(L4).

Step	Goal	How it is resolved	Result
1	Approved(L4)	Only P6 and P7 conclude Approved, and both need MgrApproved(L4).	subgoal MgrApproved
2	MgrApproved(L4) by P2	Needs Eligible(Karan, L4), which only P1 concludes. P1 needs Confirmed(Karan), but no such fact is stored, so ¬Confirmed(Karan) holds by the closed-world assumption.	× fails
3	MgrApproved(L4) by P3	Needs Emergency(L4). No such fact.	× fails
4	MgrApproved(L4) by P4	Needs Sick(L4). No such fact.	× fails
5	Approved(L4)	Subgoal MgrApproved(L4) cannot be proved.	× fails
6	Denied(L4)	P10: SubmittedBy(L4, Karan) F14, ¬Confirmed(Karan) (no fact stored), ¬Sick(L4), ¬Emergency(L4).	✓ proved

Answer: no. L4 is denied because Karan is still on probation, even though notice, coverage and balance are all fine. By P13, Notified(Karan, L4).

Query 5: Neha's request L5 and Asha's request L6 both fall in the project freeze (F36, F37). What is the outcome of each?

Method: forward chaining on both requests, comparing which rules fire.

Request	Step	New fact	Rule	Because of
L5	1	Eligible(Neha, L5)	P1	Confirmed(Neha), HasBalance(Neha, L5)
L5	2	(P2 blocked)		InFreeze(L5) is true, so P2 cannot give MgrApproved(L5)
L5	3	Denied(L5)	P8	InFreeze(L5) F36, ¬Emergency(L5), ¬Sick(L5)
L6	1	Eligible(Asha, L6)	P1	Confirmed(Asha), HasBalance(Asha, L6) F27
L6	2	(P2 blocked)		InFreeze(L6) is true, so P2 cannot give MgrApproved(L6)
L6	3	MgrApproved(L6)	P3	SubmittedBy(L6, Asha), Emergency(L6) F40, HasBalance(Asha, L6) F27 (notice, coverage and freeze are ignored)
L6	4	Approved(L6)	P7	MgrApproved(L6), Duration(L6, 1) F22, 1 ≤ 5
L6	5	(P8 blocked)		P8 contains ¬Emergency(L6), which is false

Answer: L5 is denied and L6 is approved. The freeze facts are the same kind, but the outcomes differ because of the emergency exception, which the rules handle without any extra code.

Summary of all six requests (hand derivation = program output; P13 also fires for every request):

Request	Employee	Type / duration	Rules fired	Key reason	Outcome
L1	Asha	Planned, 2 days	P1, P2, P7	Eligible, notice, coverage, no freeze	Approved
L2	Ravi	Planned, 8 days	P1, P2, P5, P6	Over 5 days, HR approved	Approved
L3	Meera	Sick, 3 days	P4, P7	Certificate attached; balance not needed	Approved
L4	Karan	Planned, 3 days	P10	On probation	Denied
L5	Neha	Planned, 3 days	P1, P8	Project freeze	Denied
L6	Asha	Emergency, 1 day	P1, P3, P7	Emergency exception ignores the freeze	Approved

B.4 Analyse (1 mark)

How predicate logic improves on propositional logic here

Predicate logic let one set of 13 rules decide every leave request. Propositional logic needs a fresh set of 16 symbols for each request, and it cannot check the numeric, relational and "some person" parts of the rules at all.

What the workplace needs	Propositional logic	Predicate logic
Many employees and requests	Symbols describe one request. Six requests need 13 × 6 = 78 rule copies and about 100 symbols (Conf_Asha, Bal_L1, ...).	One rule such as P2 covers every employee and request through variables. A new employee is just new facts.
Relationships between objects	No way to say "this request belongs to that person".	SubmittedBy(r, x) and Manages(m, x) link requests, employees and managers (MemberOf(x, t) does the same for teams in the extension rules).
Numeric tests (5-day HR threshold)	Long is decided outside the logic, so part of the policy is hidden.	Duration(r, d) ∧ d > 5 writes the threshold inside the rule (P5, P7).

What the workplace needs	Propositional logic	Predicate logic
"Some HR approver exists"	Cannot be written with fixed propositions.	P12 uses ¬∃h, and the program checks it.
Rules across several requests	Impossible.	The extension rules P14 (no overlapping approved leaves) and P15 (team coverage, using "some teammate is on leave") can be written because they compare two or more requests. They are specified but not run, as the KB has no Overlaps or MemberOf facts.
Open questions	Only yes or no about one fixed request.	A query with a variable returns bindings: ∃r Approved(r) gives r = L1, L2, L3, L6.

How well the knowledge base solves the problem

All six test requests, which cover short leave, long leave with HR escalation, sick leave, probation, freeze and the emergency exception, give the outcome that the written policy requires.

- Each decision comes with a proof (the chain of rules and facts), which supports explainability and audit.
- The rules are modular, so HR can read and change one rule at a time.
- The exceptions (emergency and sick leave) are expressed by explicit conditions, and no approval and denial rule fire together on the test data.

Missing knowledge and limitations

- **Closed world.** A fact that is not stored is treated as false. If HR has simply not yet reviewed a long leave, P12 denies it, when in reality it should be "pending". The KB has no "unknown" or "pending" state.
- **Simplified balance and dates.** HasBalance is a yes/no fact. A real check is Balance(x, b) ∧ b ≥ d and depends on accrual and carry-forward. Overlaps, Covered and InFreeze are supplied as facts rather than computed from calendars, holidays and weekends.
- **Cross-request rules not implemented.** P14 and P15 are written but not run. To run them the program would need MemberOf and Overlaps facts. It would also need an ordering (for example, the earlier request wins), because otherwise two overlapping approved requests would deny each other under P14. P15 would need a positive predicate such as Uncovered(r) in place of ¬Covered(r), since forward chaining derives only positive facts.
- **Backward chaining is a proof trace.** The program explains a goal by reading back the derivations stored during forward chaining. It does not run an independent goal-directed search, so it adds explanation, not extra reasoning power.
- **Limited leave types.** Maternity and paternity, compensatory off, loss-of-pay, half-day and bereavement leave are not modelled, and each has its own statutory rules.
- **No human judgement or overrides.** Managers sometimes approve an exception (for example a notice shortfall for a genuine reason). The KB has no exception or priority mechanism.
- **No absent approvers or time limits.** There is no delegation and no auto-escalation if a manager does not respond within 48 hours.
- **Data quality.** Wrong certificate or balance data produces a wrong decision with a confident proof.

B.5 Reflect (1 mark)

Practical constraints on deployment

- **Incomplete and stale knowledge.** Facts such as balance, team calendar and freeze windows live in different systems (HRMS, project tools). Some are missing or outdated, so the KB would reason from partial data.
- **Evolving business rules.** Leave policy changes with each financial year, new labour regulations and client commitments. The rules need an owner, version control and a test for each rule, or the KB will enforce last year's policy.

- **Privacy.** Sick-leave requests carry health information. Under India's Digital Personal Data Protection Act, 2023 such data must be minimised and protected. The KB should only store a yes/no "certificate valid" flag, not the diagnosis, and access to proofs should be restricted.
- **Scalability.** Thousands of employees and requests need an efficient engine (Datalog or Prolog style, or a Rete-based rule engine) with the facts in a database. Full first-order reasoning is not guaranteed to terminate, so the rules must stay in a restricted Horn-clause form.
- **Integration and trust.** The KB must connect to the HRMS, and employees and managers must understand automated outcomes. Unfair or unexplained denials would reduce acceptance.

The single most important improvement

Use the KB as a decision-support tool with a "Pending / Needs review" state, and never let it auto-deny.

Today the KB treats missing information as false, so a missing certificate record, a delayed HR response or a stale balance can produce a wrongful denial (see the P12 issue). The improvement has three parts: (i) add a third outcome, Pending, whenever a required fact is unknown rather than false; (ii) send every denial and every Pending case to the manager or HR together with the proof trace for a human decision; (iii) allow automatic action only for clear approvals.

Leave is linked to statutory rights and to employee trust, so a wrong automatic denial is costly, whereas a wrong approval is easily reversed. Real HR data is almost always incomplete, so the closed-world assumption is the biggest source of error in this KB, and fixing it gives more value than adding new rules. Keeping a human in the loop also handles the cases the rules cannot express, and the proof traces make the reviewer's work fast. Other improvements, such as HRMS integration and rule versioning, build on this one.

What I learned

- Writing the policy as logic forced exact definitions, for example that sick and emergency leave are exempt from some checks but not from others.
- The freeze facts for L5 and L6 gave two different outcomes (L5 denied, L6 approved). The exception was only a condition inside the rules, with no special code.
- Negation (¬InFreeze, ¬Confirmed) made the rules easy to read, but it makes the closed-world assumption matter: the safety question moves into the facts.
- In a real deployment the facts would come from the HR system instead of being typed in, and every decision would be logged with its proof.

Appendix A: Python implementation

I coded the knowledge base in Python 3 using only the standard library (file leave_kb.py). The 41 facts (F1 to F41) are stored as tuples; Confirmed(Karan) is simply not stored. Forward chaining applies the 13 rules P1 to P13 until no new fact appears; each derived fact records the rule and premises that produced it, which gives the explanations in B.3.5, and the backward-chaining proof trace prints the tree of rules and facts behind a goal, using the derivations recorded during forward chaining (it is a trace, not a separate goal-directed search). The run derived 21 new facts (41 base + 21 derived = 62 in total). The fact tuples and the helper functions (has, requester, duration, hr_approved) come before this code in leave_kb.py and are not reproduced here.

Rule and chaining code (rules P1 to P13, forward chaining, backward chaining):

```
# ---------------------------------------------------------------------
# 3. INFERENCE RULES P1 - P13
# Each rule is a function rule(kb, r) -> (derived_fact, premises) or None
# ---------------------------------------------------------------------
def P1(kb, r): # Confirmed(x) ^ SubmittedBy(r,x) ^ HasBalance(x,r) -> Eligible(x,r)
    x = requester(kb, r)
    if x and has(kb, "Confirmed", x) and has(kb, "HasBalance", x, r):
        return ("Eligible", x, r), [("Confirmed", x), ("SubmittedBy", r, x),
                                    ("HasBalance", x, r)]
def P2(kb, r): # Eligible ^ NoticeGiven ^ Covered ^ ~InFreeze -> MgrApproved
    x = requester(kb, r)
    if (x and has(kb, "Eligible", x, r) and has(kb, "NoticeGiven", r)
            and has(kb, "Covered", r) and not has(kb, "InFreeze", r)):
        return ("MgrApproved", r), [("Eligible", x, r), ("NoticeGiven", r),
                                    ("Covered", r), "not InFreeze(%s)" % r]
def P3(kb, r): # SubmittedBy ^ Emergency ^ HasBalance -> MgrApproved
    x = requester(kb, r)
    if x and has(kb, "Emergency", r) and has(kb, "HasBalance", x, r):
        return ("MgrApproved", r), [("SubmittedBy", r, x), ("Emergency", r),
                                    ("HasBalance", x, r)]
def P4(kb, r): # Sick ^ CertAttached -> MgrApproved
    if has(kb, "Sick", r) and has(kb, "CertAttached", r):
        return ("MgrApproved", r), [("Sick", r), ("CertAttached", r)]
def P5(kb, r): # MgrApproved ^ Duration(r,d) ^ d>5 -> NeedsHR
    d = duration(kb, r)
    if has(kb, "MgrApproved", r) and d is not None and d > 5:
        return ("NeedsHR", r), [("MgrApproved", r), ("Duration", r, d), "%d > 5" % d]
def P6(kb, r): # NeedsHR ^ HRManager(h) ^ HRApproves(h,r) -> Approved
    h = hr_approved(kb, r)
    if has(kb, "NeedsHR", r) and h:
        return ("Approved", r), [("NeedsHR", r), ("HRManager", h),
                                 ("HRApproves", h, r)]
def P7(kb, r): # MgrApproved ^ Duration(r,d) ^ d<=5 -> Approved
    d = duration(kb, r)
    if has(kb, "MgrApproved", r) and d is not None and d <= 5:
        return ("Approved", r), [("MgrApproved", r), ("Duration", r, d), "%d <= 5" % d]
def P8(kb, r): # (InFreeze v ~Covered v ~NoticeGiven) ^ ~Emergency ^ ~Sick -> Denied
    if (not has(kb, "Emergency", r) and not has(kb, "Sick", r)):
        why = []
        if has(kb, "InFreeze", r):
            why.append(("InFreeze", r))
        if not has(kb, "Covered", r):
            why.append("not Covered(%s)" % r)
        if not has(kb, "NoticeGiven", r):
            why.append("not NoticeGiven(%s)" % r)
        if why:
            return ("Denied", r), why + ["not Emergency", "not Sick"]
def P9(kb, r): # SubmittedBy ^ ~HasBalance ^ ~Sick -> Denied
    x = requester(kb, r)
    if x and not has(kb, "HasBalance", x, r) and not has(kb, "Sick", r):
        return ("Denied", r), [("SubmittedBy", r, x),
                               "not HasBalance(%s,%s)" % (x, r), "not Sick"]
def P10(kb, r): # SubmittedBy ^ ~Confirmed ^ ~Sick ^ ~Emergency -> Denied
    x = requester(kb, r)
```

```
    if (x and not has(kb, "Confirmed", x) and not has(kb, "Sick", r)
            and not has(kb, "Emergency", r)):
        return ("Denied", r), [("SubmittedBy", r, x), "not Confirmed(%s)" % x,
                               "not Sick", "not Emergency"]
def P11(kb, r): # Sick ^ ~CertAttached -> Denied
    if has(kb, "Sick", r) and not has(kb, "CertAttached", r):
        return ("Denied", r), [("Sick", r), "not CertAttached(%s)" % r]
def P12(kb, r): # NeedsHR ^ ~exists h(HRManager(h) ^ HRApproves(h,r)) -> Denied
    if has(kb, "NeedsHR", r) and not hr_approved(kb, r):
        return ("Denied", r), [("NeedsHR", r), "no HR manager has approved"]
def P13(kb, r): # SubmittedBy ^ (Approved v Denied) -> Notified(x,r)
    x = requester(kb, r)
    if x and (has(kb, "Approved", r) or has(kb, "Denied", r)):
        out = "Approved" if has(kb, "Approved", r) else "Denied"
        return ("Notified", x, r), [("SubmittedBy", r, x), (out, r)]
RULES = [("P1", P1), ("P2", P2), ("P3", P3), ("P4", P4), ("P5", P5),
         ("P6", P6), ("P7", P7), ("P8", P8), ("P9", P9), ("P10", P10),
         ("P11", P11), ("P12", P12), ("P13", P13)]
# ---------------------------------------------------------------------
# 4. FORWARD CHAINING (apply rules until nothing new is derived)
# ---------------------------------------------------------------------
def forward_chain(base_facts):
    kb = set(base_facts)
    proof = {}  # derived fact -> (rule name, premises)
    changed = True
    while changed:
        changed = False
        for r in REQUESTS:
            for name, rule in RULES:
                res = rule(kb, r)
                if res and res[0] not in kb:
                    fact, premises = res
                    kb.add(fact)
                    proof[fact] = (name, premises)
                    changed = True
    return kb, proof
# ---------------------------------------------------------------------
# 5. BACKWARD CHAINING (start from goal, look for supporting facts/rules)
# Uses the proof table built by forward chaining to print the tree.
# ---------------------------------------------------------------------
def fmt(f):
    if isinstance(f, str):
        return f
    return "%s(%s)" % (f[0], ", ".join(str(a) for a in f[1:]))
def backward_chain(goal, kb, proof, depth=0):
    """Return True/False and print the proof tree for `goal`."""
    pad = "  " * depth
    if goal in proof:
        rule, premises = proof[goal]
        print("%s%s <- derived by %s" % (pad, fmt(goal), rule))
        for p in premises:
            if isinstance(p, tuple):
                backward_chain(p, kb, proof, depth + 1)
            else:
                print("%s  %s (true by closed-world assumption / arithmetic)"
                      % (pad, p))
        return True
    if goal in kb:
        print("%s%s <- given FACT" % (pad, fmt(goal)))
        return True
    print("%s%s <- cannot be proved (FAILS)" % (pad, fmt(goal)))
    return False
```

The program running. The screenshots below were taken in the VS Code terminal. The command is `python leave_kb.py`. Each tree shows a derived fact with the rule that produced it ("derived by P7"), and the facts or comparisons it came from indented below it.

```
C:\Users\vansh\Downloads>python leave_kb.py
Base facts: 41   |   Derived facts: 21   |   Total: 62

================================================================
QUERY 1: Is Asha's request L1 (2 days) approved?
================================================================
Approved(L1)   <- derived by P7
    MgrApproved(L1)   <- derived by P2
        Eligible(Asha, L1)   <- derived by P1
            Confirmed(Asha)   <- given FACT
            SubmittedBy(L1, Asha)   <- given FACT
            HasBalance(Asha, L1)   <- given FACT
        NoticeGiven(L1)   <- given FACT
        Covered(L1)   <- given FACT
        not InFreeze(L1)   (true by closed-world assumption / arithmetic)
    Duration(L1, 2)   <- given FACT
    2 <= 5   (true by closed-world assumption / arithmetic)
ANSWER: YES
Notified(Asha, L1)   <- derived by P13
    SubmittedBy(L1, Asha)   <- given FACT
    Approved(L1)   <- derived by P7
        MgrApproved(L1)   <- derived by P2
            Eligible(Asha, L1)   <- derived by P1
                Confirmed(Asha)   <- given FACT
                SubmittedBy(L1, Asha)   <- given FACT
                HasBalance(Asha, L1)   <- given FACT
            NoticeGiven(L1)   <- given FACT
            Covered(L1)   <- given FACT
```

Figure A1: Query 1, the derivation of Approved(L1) and Notified(Asha, L1).

```
Problems 160   Output   Debug Console   Terminal   Ports   Spell Checker 46   Azure

================================================================
QUERY 2: Ravi's request L2 (8 days) - approved? who must approve?
================================================================
Approved(L2)   <- derived by P6
    NeedsHR(L2)   <- derived by P5
        MgrApproved(L2)   <- derived by P2
            Eligible(Ravi, L2)   <- derived by P1
                Confirmed(Ravi)   <- given FACT
                SubmittedBy(L2, Ravi)   <- given FACT
                HasBalance(Ravi, L2)   <- given FACT
            NoticeGiven(L2)   <- given FACT
            Covered(L2)   <- given FACT
            not InFreeze(L2)   (true by closed-world assumption / arithmetic)
        Duration(L2, 8)   <- given FACT
        8 > 5   (true by closed-world assumption / arithmetic)
    HRManager(Anita)   <- given FACT
    HRApproves(Anita, L2)   <- given FACT
ANSWER: YES | NeedsHR(L2) = True | Manager = Suresh, HR = Anita
What-if F42 is missing -> Denied(L2) = True (P12 fires)

================================================================
QUERY 3: Is Meera's sick leave L3 approved with no balance fact?
================================================================
Approved(L3)   <- derived by P7
    MgrApproved(L3)   <- derived by P4
        Sick(L3)   <- given FACT
        CertAttached(L3)   <- given FACT
```

Figure A2: Query 2 (L2 needs HR approval; what-if the HRApproves fact is missing) and the start of Query 3.

```
Problems 160   Output   Debug Console   Terminal   Ports   Spell Checker 46   Azure

QUERY 3: Is Meera's sick leave L3 approved with no balance fact?
================================================================
Approved(L3)   <- derived by P7
    MgrApproved(L3)   <- derived by P4
        Sick(L3)   <- given FACT
        CertAttached(L3)   <- given FACT
    Duration(L3, 3)   <- given FACT
    3 <= 5   (true by closed-world assumption / arithmetic)
ANSWER: YES | Denied(L3) = False

================================================================
QUERY 4: Can Karan's request L4 be approved? If not, why?
================================================================
Goal Approved(L4):
Approved(L4)   <- cannot be proved (FAILS)
Goal MgrApproved(L4):
MgrApproved(L4)   <- cannot be proved (FAILS)
Goal Denied(L4):
Denied(L4)   <- derived by P10
    SubmittedBy(L4, Karan)   <- given FACT
    not Confirmed(Karan)   (true by closed-world assumption / arithmetic)
    not Sick   (true by closed-world assumption / arithmetic)
    not Emergency   (true by closed-world assumption / arithmetic)
ANSWER: NO (Karan is on probation -> P10)

================================================================
QUERY 5: L5 (Neha) and L6 (Asha) both fall in the project freeze
================================================================
```

Figure A3: Query 3 (sick leave approved), Query 4 (Karan's L4 not provable, denied by P10) and the start of Query 5.

```
Request L5
Approved(L5)   <- cannot be proved (FAILS)
Denied(L5)   <- derived by P8
    InFreeze(L5)   <- given FACT
    not Emergency   (true by closed-world assumption / arithmetic)
    not Sick   (true by closed-world assumption / arithmetic)
Outcome of L5: DENIED

Request L6
Approved(L6)   <- derived by P7
    MgrApproved(L6)   <- derived by P3
        SubmittedBy(L6, Asha)   <- given FACT
        Emergency(L6)   <- given FACT
        HasBalance(Asha, L6)   <- given FACT
    Duration(L6, 1)   <- given FACT
    1 <= 5   (true by closed-world assumption / arithmetic)
Denied(L6)   <- cannot be proved (FAILS)
Outcome of L6: APPROVED

================================================================
SUMMARY OF ALL SIX REQUESTS
================================================================
Req  Emp     Type         Days      Rules used              Outcome
L1   Asha    Planned      2         P1,P2,P7                Approved
L2   Ravi    Planned      8         P1,P2,P5,P6             Approved
L3   Meera   Sick         3         P4,P7                   Approved
L4   Karan   Planned      3         P10                     Denied
```

Figure A4: Query 5 (L5 denied, L6 approved) and the summary of all six requests.

```
Problems 160   Output   Debug Console   Terminal   Ports   Spell Checker 46   Azure

Denied(L6)   <- cannot be proved (FAILS)
Outcome of L6: APPROVED

================================================================
SUMMARY OF ALL SIX REQUESTS
================================================================
Req  Emp     Type         Days      Rules used              Outcome
L1   Asha    Planned      2         P1,P2,P7                Approved
L2   Ravi    Planned      8         P1,P2,P5,P6             Approved
L3   Meera   Sick         3         P4,P7                   Approved
L4   Karan   Planned      3         P10                     Denied
L5   Neha    Planned      3         P1,P8                   Denied
L6   Asha    Emergency    1         P1,P3,P7                Approved

================================================================
NOTIFICATIONS (P13)
================================================================
Notified(Asha, L1) = True
Notified(Ravi, L2) = True
Notified(Meera, L3) = True
Notified(Karan, L4) = True
Notified(Neha, L5) = True
Notified(Asha, L6) = True

C:\Users\vansh\Downloads>
```

Figure A5: Summary table and the P13 notifications for all six requests.